Table of Contents

ИНСТРУКЦИЯ ПО ВНЕДРЕНИЮ

CK CORE LAYER SPEC — Dynastic Simulation Engine (v0.1)

Проект: ENIGMA v0.5.3.1.9 **Документ:** Спецификация и инструкция по созданию CK-style layer **Тип:** Техническое задание на добавление новых систем (не refactor, а additive layer) **Предусловие:** Kernel Isolation Repair v0.1 применён, SUPERBOX causal drift = 0%

Цель: превратить deterministic kernel в dynastic simulation engine. После применения можно строить CK-style gameplay: multi-generational play, lineage inheritance, succession, world history.

Принцип: additive systems. Не переписываем существующий код. Добавляем новые сущности поверх deterministic kernel.

---

0. Контекст: что строим и почему

0.1. Текущее состояние (ДО CK CORE LAYER)

После Kernel Isolation Repair: - ✅ Deterministic kernel (tick = causal spine, RNG seeded) - ✅ Replay determinism (0% causal drift) - ❌ No lineage (NPC не имеет родителей/детей) - ❌ No aging (NPC не стареет) - ❌ No succession (смерть = permanent, наследников нет) - ❌ No world history (только in-memory L1Chronicle per-NPC) - ❌ No cross-NPC causal graph (только player→NPC отношения)

0.2. Что CK CORE LAYER добавляет

Сущность	Назначение	Persistence
Lineage	parent→child inheritance graph	SQLite (new table)
Aging	NPC стареет по seconds (physics layer)	в NPCState
Birth event	spawn нового NPC от родителей	WorldChronicle
Death → succession	heir наследует traits покойного	WorldChronicle + Lineage
WorldChronicle	cross-NPC, cross-generation event log	SQLite (new table)

Сущность	Назначение	Persistence
Cross-NPC causal graph	NPC→NPC отношения (не только player→NPC)	SQLite (extends RelationshipStore)

0.3. Архитектурные принципы (не нарушать)

1. **Tick = causal spine.** Все CK events (birth, death, succession) — tick-indexed в WorldChronicle.
2. **Seconds = physical layer.** Aging — seconds-based (age += elapsed_seconds / SECONDS_PER_YEAR).
3. **Kernel never depends from seconds.** Lineage inheritance — deterministic function of (parent_L0, tick, rng).
4. **Additive only.** Не меняем существующий DecisionHub, StateApplicator, LifeEngine. Добавляем новые сервисы.
5. **L1 vs WorldChronicle — не пересекаются.** L1 = subjective cognition (per-NPC). WorldChronicle = objective truth (global).

---

1. Порядок внедрения (обязательный)

STEP 1 → world_chronicle.py (NEW — foundation, global event log)
STEP 2 → lineage.py (NEW — parent→child graph)
STEP 3 → aging.py (NEW — seconds-based age progression)
STEP 4 → birth_generator.py (NEW — spawn NPC from parents)
STEP 5 → succession_pipeline.py (NEW — death → heir inheritance)
STEP 6 → cross_npc_relationships.py (NEW — NPC→NPC dynamic relationships)
STEP 7 → event_types.py (MODIFY — добавить BIRTH, DEATH, etc.)
STEP 8 → npc_state.py (MODIFY — добавить lineage, age fields)
STEP 9 → npc_loader.py (MODIFY — загружать lineage fields)
STEP 10 → tick_orchestrator.py (MODIFY — вызывать aging, succession)
STEP 11 → validation tests (NEW — CK determinism tests)

Зависимости: - STEP 2-6 требуют STEP 1 (нужен WorldChronicle) - STEP 4-5 требуют STEP 2 (нужен Lineage) - STEP 8-9 требуют STEP 2-3 (нужны типы для сериализации) - STEP 10 требует STEP 1-6 (оркестрация) - STEP 11 требует всех

Откат: каждый STEP — independently testable. Можно откатывать по одному.

---

2. STEP 1 — backend/app/services/world/world_chronicle.py (NEW FILE)

2.1. Назначение

Global, cross-NPC, cross-generation event log. Хранит objective truth: births, deaths, marriages, wars, inheritance, faction changes. **Не subjective** (это L1Chronicle). **Не per-NPC** (это global).

2.2. Архитектурная граница

Слой	Что хранит	Scope	Indexed by
L1Chronicle	drift events (subjective cognition)	per-NPC	tick
WorldChronicle	**world events (objective truth)**	**global**	**tick**
SQLiteStore.event_memories	NPC memories (what NPC remembers)	per-NPC	day

Правило: L1 = «что NPC пережил». WorldChronicle = «что произошло в мире». NPC может не помнить WorldChronicle event (если не был свидетелем), но WorldChronicle остаётся фактом.

2.3. Создать файл

Путь: backend/app/services/world/world_chronicle.py

Содержимое:

```
"""
path: backend/app/services/world/world_chronicle.py
Назначение: Global, cross-NPC, cross-generation event log.
            Objective truth layer (vs L1Chronicle = subjective cognition).
Зависимости: app.services.memory.sqlite_store
Основные сущности: WorldChronicle, WorldEvent

АРХИТЕКТУРНЫЙ ПРИНЦИП:
    L1Chronicle = subjective (per-NPC, drift)
    WorldChronicle = objective (global, facts)
    Они НЕ пересекаются.

PERSISTENCE:
    SQLite (durable, cross-session).
    Tick-indexed for replay determinism.

ИСПОЛЬЗОВАНИЕ:
    chronicle = WorldChronicle(store=sqlite_store, campaign_id="open_road")
```

```
    chronicle.log_event(WorldEvent(
        type=WorldEventType.BIRTH,
        tick=1000,
        actor_id="maid_lusya_child_1",
        payload={"parents": ["maid_lusya", "guard_borko"], "inherited_traits": {...}},
    ))
    events = chronicle.query(tick_from=900, tick_to=1100)
"""

from __future__ import annotations

import json
import logging
from dataclasses import dataclass, field, asdict
from enum import Enum
from typing import Any, Dict, List, Optional
from uuid import uuid4

logger = logging.getLogger(__name__)


class WorldEventType(str, Enum):
    """Типы objective world events."""
    # ── NPC lifecycle ──────────────────────────────────────────
    BIRTH = "birth"                 # NPC родился (от родителей или spawn)
    DEATH = "death"                 # NPC умер (естественная смерть или
убийство)
    MARRIAGE = "marriage"           # NPC вступил в брак
    DIVORCE = "divorce"             # NPC развёлся
    # ── Lineage ────────────────────────────────────────────────
    INHERITANCE = "inheritance"     # NPC унаследовал traits от покойного
родителя
    SUCCESSION = "succession"       # NPC занял место покойного (heir)
    # ── Faction ────────────────────────────────────────────────
    FACTION_JOINED = "faction_joined"
    FACTION_LEFT = "faction_left"
    FACTION_WAR_DECLARED = "faction_war_declared"
    FACTION_PEACE_SIGNED = "faction_peace_signed"
    # ── Cross-NPC ──────────────────────────────────────────────
    NPC_KILLED_NPC = "npc_killed_npc"   # NPC убил другого NPC
    NPC_SAVED_NPC = "npc_saved_npc"     # NPC спас другого NPC
    NPC_BETRAYED_NPC = "npc_betrayed_npc"
    # ── World ──────────────────────────────────────────────────
    LOCATION_DISCOVERED = "location_discovered"
    SETTLEMENT_FOUNDED = "settlement_founded"
    SETTLEMENT_DESTROYED = "settlement_destroyed"


@dataclass(frozen=True)
```

```python
class WorldEvent:
    """Objective world event. Immutable after creation.

    Tick-indexed for replay determinism.
    Payload — structured, type-dependent.
    """
    id: str  # UUID
    type: WorldEventType
    tick: int  # causal tick (NOT seconds)
    actor_id: str  # NPC or faction or "world"
    target_id: Optional[str]  # affected NPC/faction (if any)
    payload: Dict[str, Any]  # type-specific structured data
    witness_ids: tuple = ()  # NPCs who witnessed (for memory propagation)

    @classmethod
    def create(
        cls,
        type: WorldEventType,
        tick: int,
        actor_id: str,
        payload: Dict[str, Any],
        target_id: Optional[str] = None,
        witness_ids: Optional[List[str]] = None,
    ) -> "WorldEvent":
        return cls(
            id=str(uuid4()),
            type=type,
            tick=tick,
            actor_id=actor_id,
            target_id=target_id,
            payload=payload,
            witness_ids=tuple(witness_ids or []),
        )


class WorldChronicle:
    """Global objective event log.

    Persistence: SQLite (durable).
    Indexing: by tick (for replay), by type (for queries), by actor (for lineage).
    """

    TABLE_NAME = "world_chronicle_events"

    def __init__(self, store=None, campaign_id: str = ""):
        """
        Args:
            store: SQLiteStore instance. If None — in-memory only (tests only).
            campaign_id: namespace for events.
        """
```

```
        self._store = store
        self._campaign_id = campaign_id
        self._cache: List[WorldEvent] = []
        self._loaded = False

    def _ensure_schema(self) -> None:
        """Создать таблицу если не существует."""
        if self._store is None:
            return
        self._store.execute(f"""
            CREATE TABLE IF NOT EXISTS {self.TABLE_NAME} (
                id TEXT PRIMARY KEY,
                campaign_id TEXT NOT NULL,
                type TEXT NOT NULL,
                tick INTEGER NOT NULL,
                actor_id TEXT NOT NULL,
                target_id TEXT,
                payload_json TEXT NOT NULL,
                witness_ids_json TEXT DEFAULT '[]'
            )
        """)
        self._store.execute(f"""
            CREATE INDEX IF NOT EXISTS idx_world_chronicle_tick
            ON {self.TABLE_NAME}(campaign_id, tick)
        """)
        self._store.execute(f"""
            CREATE INDEX IF NOT EXISTS idx_world_chronicle_actor
            ON {self.TABLE_NAME}(campaign_id, actor_id, tick)
        """)
        self._store.execute(f"""
            CREATE INDEX IF NOT EXISTS idx_world_chronicle_type
            ON {self.TABLE_NAME}(campaign_id, type, tick)
        """)

    def _ensure_loaded(self) -> None:
        """Lazy load from SQLite."""
        if self._loaded or self._store is None:
            return
        self._ensure_schema()
        try:
            rows = self._store.query(
                f"SELECT * FROM {self.TABLE_NAME} WHERE campaign_id = ?
ORDER BY tick ASC",
                (self._campaign_id,)
            )
            for row in rows:
                event = WorldEvent(
                    id=row["id"],
                    type=WorldEventType(row["type"]),
                    tick=row["tick"],
```

```
                actor_id=row["actor_id"],
                target_id=row["target_id"],
                payload=json.loads(row["payload_json"]),
                witness_ids=tuple(json.loads(row["witness_ids_json"] or "[]")),
            )
            self._cache.append(event)
        self._loaded = True
        logger.debug(
            f"[WORLD_CHRONICLE] loaded {len(self._cache)} events "
            f"for campaign={self._campaign_id}"
        )
    except Exception as e:
        logger.error(f"[WORLD_CHRONICLE] CRITICAL: failed to load: {e}")
        raise

def log_event(self, event: WorldEvent) -> None:
    """Log objective world event. Atomic."""
    self._ensure_loaded()
    self._cache.append(event)

    if self._store is not None:
        try:
            self._store.execute(
                f"INSERT INTO {self.TABLE_NAME} "
                f"(id, campaign_id, type, tick, actor_id, target_id, payload_json,
witness_ids_json) "
                f"VALUES (?, ?, ?, ?, ?, ?, ?, ?)",
                (
                    event.id,
                    self._campaign_id,
                    event.type.value,
                    event.tick,
                    event.actor_id,
                    event.target_id,
                    json.dumps(event.payload, ensure_ascii=False),
                    json.dumps(list(event.witness_ids), ensure_ascii=False),
                )
            )
        except Exception as e:
            logger.error(f"[WORLD_CHRONICLE] CRITICAL: failed to persist event
{event}: {e}")
            raise

def query(
    self,
    tick_from: int = 0,
    tick_to: Optional[int] = None,
    type_filter: Optional[WorldEventType] = None,
    actor_filter: Optional[str] = None,
) -> List[WorldEvent]:
```

```
    """Query events by tick range, type, actor."""
    self._ensure_loaded()
    result = []
    for event in self._cache:
        if event.tick < tick_from:
            continue
        if tick_to is not None and event.tick > tick_to:
            continue
        if type_filter is not None and event.type != type_filter:
            continue
        if actor_filter is not None and event.actor_id != actor_filter:
            continue
        result.append(event)
    return result

def query_lineage(self, npc_id: str) -> List[WorldEvent]:
    """Все events связанные с NPC (как actor или target)."""
    self._ensure_loaded()
    return [
        e for e in self._cache
        if e.actor_id == npc_id or e.target_id == npc_id
        or npc_id in e.witness_ids
    ]

def get_npc_history(self, npc_id: str) -> Dict[str, Any]:
    """Полная история NPC: birth, death, marriages, inheritance, etc."""
    events = self.query_lineage(npc_id)
    history = {
        "birth": None,
        "death": None,
        "marriages": [],
        "inheritance_received": [],
        "inheritance_given": [],
        "succession_events": [],
        "faction_events": [],
        "cross_npc_events": [],
    }
    for e in events:
        if e.type == WorldEventType.BIRTH and e.actor_id == npc_id:
            history["birth"] = e
        elif e.type == WorldEventType.DEATH and e.actor_id == npc_id:
            history["death"] = e
        elif e.type == WorldEventType.MARRIAGE and e.actor_id == npc_id:
            history["marriages"].append(e)
        elif e.type == WorldEventType.INHERITANCE and e.target_id == npc_id:
            history["inheritance_received"].append(e)
        elif e.type == WorldEventType.INHERITANCE and e.actor_id == npc_id:
            history["inheritance_given"].append(e)
        elif e.type == WorldEventType.SUCCESSION and e.actor_id == npc_id:
            history["succession_events"].append(e)
```

```
        elif e.type in (WorldEventType.FACTION_JOINED,
WorldEventType.FACTION_LEFT):
            history["faction_events"].append(e)
        elif e.type in (WorldEventType.NPC_KILLED_NPC,
WorldEventType.NPC_SAVED_NPC, WorldEventType.NPC_BETRAYED_NPC):
            history["cross_npc_events"].append(e)
    return history
```

2.4. Регрессионный тест

Путь: backend/tests/test_world_chronicle.py

```
"""Тесты WorldChronicle persistence и queries."""
import pytest
from app.services.world.world_chronicle import WorldChronicle, WorldEvent,
WorldEventType


def test_log_and_query_event(tmp_path):
    """Event логируется и находится по query."""
    from app.services.memory.sqlite_store import SQLiteStore
    store = SQLiteStore(str(tmp_path / "test.db"))
    chronicle = WorldChronicle(store=store, campaign_id="test")

    event = WorldEvent.create(
        type=WorldEventType.BIRTH,
        tick=100,
        actor_id="child_1",
        payload={"parents": ["maid_lusya", "guard_borko"]},
    )
    chronicle.log_event(event)

    # New instance — simulates restart
    chronicle2 = WorldChronicle(store=store, campaign_id="test")
    events = chronicle2.query(tick_from=0)

    assert len(events) == 1
    assert events[0].type == WorldEventType.BIRTH
    assert events[0].actor_id == "child_1"
    assert events[0].payload["parents"] == ["maid_lusya", "guard_borko"]


def test_query_by_type(tmp_path):
    """Filter by type."""
    from app.services.memory.sqlite_store import SQLiteStore
    store = SQLiteStore(str(tmp_path / "test.db"))
    chronicle = WorldChronicle(store=store, campaign_id="test")

    chronicle.log_event(WorldEvent.create(WorldEventType.BIRTH, 100, "child_1",
{}))
```

```
    chronicle.log_event(WorldEvent.create(WorldEventType.DEATH, 200, "elder_1",
{}))
    chronicle.log_event(WorldEvent.create(WorldEventType.BIRTH, 300, "child_2",
{}))

    births = chronicle.query(type_filter=WorldEventType.BIRTH)
    deaths = chronicle.query(type_filter=WorldEventType.DEATH)

    assert len(births) == 2
    assert len(deaths) == 1


def test_get_npc_history(tmp_path):
    """Полная история NPC."""
    from app.services.memory.sqlite_store import SQLiteStore
    store = SQLiteStore(str(tmp_path / "test.db"))
    chronicle = WorldChronicle(store=store, campaign_id="test")

    # Birth
    chronicle.log_event(WorldEvent.create(
        WorldEventType.BIRTH, 100, "maid_lusya",
        {"parents": ["unknown"]}
    ))
    # Marriage
    chronicle.log_event(WorldEvent.create(
        WorldEventType.MARRIAGE, 500, "maid_lusya",
        {"spouse": "guard_borko"}
    ))
    # Death
    chronicle.log_event(WorldEvent.create(
        WorldEventType.DEATH, 1000, "maid_lusya",
        {"cause": "old_age"}
    ))

    history = chronicle.get_npc_history("maid_lusya")

    assert history["birth"] is not None
    assert history["death"] is not None
    assert len(history["marriages"]) == 1
```

2.5. Критерий приёмки

```
cd backend && python -m pytest tests/test_world_chronicle.py -v
```

Все 3 теста должны проходить.

---

3. STEP 2 — backend/app/services/world/lineage.py (NEW FILE)

3.1. Назначение

Parent→child inheritance graph. Хранит genealogy NPC. Определяет, что наследует потомок от родителя.

3.2. Inheritance rules (зафиксированы)

Что наследуется	Как	Почему
drives_base (L0 seed)	100% — genetic	Базовая личность — генетика
trauma_markers	probabilistic (decayed)	Эпигенетика — травмы предков влияют, но затухают
relationship_cache bias	partial	Социальное наследство — друзья/враги семьи
faction_membership	inherited from dominant parent	Социальная преемственность
L1Chronicle	❌ NOT inherited	Индивидуальный опыт
identity_integrity	❌ NOT inherited	Индивидуальное
will_state	❌ NOT inherited	Индивидуальное
decisions	❌ NOT inherited	Индивидуальное

3.3. Создать файл

Путь: backend/app/services/world/lineage.py

Содержимое:

```
"""
path: backend/app/services/world/lineage.py
Назначение: Parent→child inheritance graph. Genealogy NPC.
Зависимости: app.services.world.world_chronicle, app.services.npc.kernel_rng
Основные сущности: Lineage, LineageRecord, InheritanceResult

АРХИТЕКТУРНЫЙ ПРИНЦИП:
    Lineage — deterministic function of (parent_L0, tick, rng).
    Inheritance — NOT continuous drift (это L1). Это discrete genetic transfer.

INHERITANCE RULES (non-negotiable):
    ✅ drives_base (L0 seed modulation) — genetic
    ✅ trauma_markers (decayed probabilistic) — epigenetic
    ✅ relationship_cache bias (partial) — social inheritance
    ✅ faction_membership (dominant parent) — social continuity
    ❌ L1Chronicle — individual experience
    ❌ identity_integrity — individual
```

```
    ❌ will_state — individual
    ❌ decisions — individual
"""

from __future__ import annotations

import logging
from dataclasses import dataclass, field
from typing import Any, Dict, List, Optional, Tuple

from app.services.npc.kernel_rng import KernelRNG
from app.services.world.world_chronicle import WorldChronicle, WorldEvent,
WorldEventType

logger = logging.getLogger(__name__)


@dataclass(frozen=True)
class LineageRecord:
    """Genealogy node: NPC + parents + children."""
    npc_id: str
    parent_ids: Tuple[str, ...]  # 0 (spawn), 1 (asexual), 2 (sexual reproduction)
    children_ids: Tuple[str, ...] = ()
    birth_tick: int = 0
    death_tick: Optional[int] = None
    generation: int = 0  # 0 = founder, 1 = first generation, etc.

    @property
    def is_founder(self) -> bool:
        """Founder = first generation, no parents."""
        return len(self.parent_ids) == 0

    @property
    def is_alive(self) -> bool:
        return self.death_tick is None


@dataclass
class InheritanceResult:
    """Что потомок наследует от родителя."""
    drives_base: Dict[str, float]  # modulated from parent
    trauma_markers: set  # probabilistic subset
    relationship_bias: Dict[str, Tuple[float, float]]  # npc_id → (trust_bias, fear_bias)
    faction_membership: Optional[str]  # dominant parent's faction
    inherited_traits: Dict[str, Any]  # metadata for chronicle


class Lineage:
    """Parent→child inheritance graph.
```

```
*Persistence: in-memory cache + WorldChronicle events (durable).
NPC genealogy reconstructed from WorldChronicle BIRTH/DEATH events.
"""

def __init__(self, chronicle: WorldChronicle):
    self._chronicle = chronicle
    self._cache: Dict[str, LineageRecord] = {}
    self._loaded = False

def _ensure_loaded(self) -> None:
    """Reconstruct lineage from WorldChronicle events."""
    if self._loaded:
        return
    # Load all BIRTH and DEATH events
    births = self._chronicle.query(type_filter=WorldEventType.BIRTH)
    deaths = self._chronicle.query(type_filter=WorldEventType.DEATH)

    # Build records
    for birth in births:
        npc_id = birth.actor_id
        parents = tuple(birth.payload.get("parents", []))
        generation = birth.payload.get("generation", 0)
        self._cache[npc_id] = LineageRecord(
            npc_id=npc_id,
            parent_ids=parents,
            birth_tick=birth.tick,
            generation=generation,
        )

    # Apply deaths
    for death in deaths:
        npc_id = death.actor_id
        if npc_id in self._cache:
            record = self._cache[npc_id]
            self._cache[npc_id] = LineageRecord(
                npc_id=record.npc_id,
                parent_ids=record.parent_ids,
                children_ids=record.children_ids,
                birth_tick=record.birth_tick,
                death_tick=death.tick,
                generation=record.generation,
            )

    # Build children lists
    for npc_id, record in self._cache.items():
        for parent_id in record.parent_ids:
            if parent_id in self._cache:
                parent = self._cache[parent_id]
                self._cache[parent_id] = LineageRecord(
```

```
                    npc_id=parent.npc_id,
                    parent_ids=parent.parent_ids,
                    children_ids=parent.children_ids + (npc_id,),
                    birth_tick=parent.birth_tick,
                    death_tick=parent.death_tick,
                    generation=parent.generation,
                )

        self._loaded = True

    def get_record(self, npc_id: str) -> Optional[LineageRecord]:
        """Get lineage record for NPC."""
        self._ensure_loaded()
        return self._cache.get(npc_id)

    def get_ancestors(self, npc_id: str, max_depth: int = 10) -> List[str]:
        """Get all ancestors up to max_depth generations."""
        self._ensure_loaded()
        ancestors = []
        visited = set()
        queue = [(npc_id, 0)]
        while queue:
            current_id, depth = queue.pop(0)
            if depth >= max_depth or current_id in visited:
                continue
            visited.add(current_id)
            record = self._cache.get(current_id)
            if record is None:
                continue
            for parent_id in record.parent_ids:
                ancestors.append(parent_id)
                queue.append((parent_id, depth + 1))
        return ancestors

    def get_descendants(self, npc_id: str, max_depth: int = 10) -> List[str]:
        """Get all descendants up to max_depth generations."""
        self._ensure_loaded()
        descendants = []
        visited = set()
        queue = [(npc_id, 0)]
        while queue:
            current_id, depth = queue.pop(0)
            if depth >= max_depth or current_id in visited:
                continue
            visited.add(current_id)
            record = self._cache.get(current_id)
            if record is None:
                continue
            for child_id in record.children_ids:
                descendants.append(child_id)
```

```
                queue.append((child_id, depth + 1))
        return descendants

    def compute_inheritance(
        self,
        parent_npc_dict: Dict[str, Any],
        tick: int,
        rng: KernelRNG,
        second_parent_dict: Optional[Dict[str, Any]] = None,
    ) -> InheritanceResult:
        """Compute what child inherits from parent(s).

        DETERMINISTIC: same (parent, tick, rng) → same inheritance.
        """
        parent_id = parent_npc_dict.get("id", parent_npc_dict.get("npc_id",
"unknown"))

        # 1. drives_base — genetic modulation
        parent_drives = dict(parent_npc_dict.get("drives", {
            "control": 0.25, "significance": 0.25, "fear": 0.25, "desire": 0.25
        }))

        if second_parent_dict:
            # Sexual reproduction — average of both parents
            second_drives = dict(second_parent_dict.get("drives", parent_drives))
            child_drives = {}
            for key in parent_drives:
                # 50/50 average + small deterministic mutation
                avg = (parent_drives.get(key, 0.25) + second_drives.get(key, 0.25)) / 2
                mutation = rng.uniform(-0.05, 0.05)  # genetic drift
                child_drives[key] = max(0.01, min(0.99, avg + mutation))
        else:
            # Asexual — clone with mutation
            child_drives = {}
            for key, value in parent_drives.items():
                mutation = rng.uniform(-0.05, 0.05)
                child_drives[key] = max(0.01, min(0.99, value + mutation))

        # Normalize (Закон Сохранения Я)
        total = sum(child_drives.values())
        if total > 0:
            for key in child_drives:
                child_drives[key] /= total

        # 2. trauma_markers — probabilistic epigenetic inheritance
        parent_traumas = set(parent_npc_dict.get("psyche", {}).get("trauma_flags",
[]))
        if second_parent_dict:
            parent_traumas |= set(second_parent_dict.get("psyche",
{}).get("trauma_flags", []))
```

```
        # 30% chance per trauma to inherit (decayed)
        inherited_traumas = set()
        for trauma in parent_traumas:
            if rng.random() < 0.3:
                inherited_traumas.add(trauma)

        # 3. relationship_bias — social inheritance
        parent_social = parent_npc_dict.get("social_stats", {})
        parent_trust = parent_social.get("trust", 0.0)
        parent_fear = parent_social.get("fear_of_player", 0.0)

        # Child inherits bias toward player (and other NPCs) from parent
        # 50% of parent's attitude, deterministic
        relationship_bias = {
            "player": (parent_trust * 0.5, parent_fear * 0.5),
        }
        # TODO: extend to NPC→NPC relationships when CrossNPCRelationships
built

        # 4. faction_membership — dominant parent
        parent_faction = parent_npc_dict.get("faction")
        if second_parent_dict and not parent_faction:
            parent_faction = second_parent_dict.get("faction")

        return InheritanceResult(
            drives_base=child_drives,
            trauma_markers=inherited_traumas,
            relationship_bias=relationship_bias,
            faction_membership=parent_faction,
            inherited_traits={
                "parent_ids": [parent_id] + ([second_parent_dict.get("id")] if
second_parent_dict else []),
                "generation": parent_npc_dict.get("generation", 0) + 1,
                "mutation_seed": rng.seed,
            },
        )
```

3.4. Критерий приёмки

```
cd backend && python -c "
from app.services.world.lineage import Lineage, InheritanceResult
from app.services.npc.kernel_rng import KernelRNG

# Test inheritance determinism
parent = {'id': 'maid_lusya', 'drives': {'control': 0.3, 'significance': 0.2, 'fear': 0.4,
'desire': 0.1}}
rng1 = KernelRNG(tick=1000, npc_id='child_1')
rng2 = KernelRNG(tick=1000, npc_id='child_1')

from app.services.world.world_chronicle import WorldChronicle
```

```
chronicle = WorldChronicle()  # in-memory
lineage = Lineage(chronicle)

result1 = lineage.compute_inheritance(parent, 1000, rng1)
result2 = lineage.compute_inheritance(parent, 1000, rng2)

assert result1.drives_base == result2.drives_base  # deterministic
print('Lineage inheritance deterministic OK')
"
```

---

4. STEP 3 — backend/app/services/world/aging.py (NEW FILE)

4.1. Назначение

NPC aging — **physics layer** (seconds-based, NOT tick-based). NPC стареет по реальному игровому времени, не по количеству решений.

4.2. Архитектурный принцип

> **Aging MUST be physics-layer only.** age += elapsed_seconds / SECONDS_PER_YEAR. Tick cannot be used for aging (fast-forward breaks it).

4.3. Создать файл

Путь: backend/app/services/world/aging.py

Содержимое:

```
"""
path: backend/app/services/world/aging.py
Назначение: NPC aging — physics layer (seconds-based, NOT tick-based).
Зависимости: app.core.constants
Основные сущности: AgingEngine, AgeStage

АРХИТЕКТУРНЫЙ ПРИНЦИП:
    Aging = physics layer (seconds).
    age += elapsed_seconds / SECONDS_PER_YEAR.
    Tick cannot be used for aging (fast-forward breaks it).

AGE STAGES:
    CHILD (0-12 game-years)    — reduced agency, learning
    TEEN (13-17)               — full agency, identity formation
    ADULT (18-60)              — full agency, reproductive
    ELDER (61-80)              — reduced physical, full mental
    ANCIENT (81+)              — high death probability
"""

from __future__ import annotations
```

```python
import logging
from dataclasses import dataclass
from enum import Enum
from typing import Dict, Optional

logger = logging.getLogger(__name__)


class AgeStage(str, Enum):
    CHILD = "child"       # 0-12
    TEEN = "teen"         # 13-17
    ADULT = "adult"       # 18-60
    ELDER = "elder"       # 61-80
    ANCIENT = "ancient"   # 81+


@dataclass
class AgeState:
    """NPC age state (physics layer)."""
    age_years: float = 25.0  # default adult
    birth_tick: int = 0  # when NPC was born (for chronicle)
    birth_seconds: float = 0.0  # game_time_seconds at birth

    @property
    def stage(self) -> AgeStage:
        if self.age_years < 13:
            return AgeStage.CHILD
        elif self.age_years < 18:
            return AgeStage.TEEN
        elif self.age_years < 61:
            return AgeStage.ADULT
        elif self.age_years < 81:
            return AgeStage.ELDER
        else:
            return AgeStage.ANCIENT

    @property
    def is_reproductive(self) -> bool:
        """Can this NPC have children?"""
        return 18 <= self.age_years <= 60

    @property
    def death_probability_per_year(self) -> float:
        """Probability of natural death per game-year."""
        if self.age_years < 60:
            return 0.001  # very low
        elif self.age_years < 80:
            return 0.02  # 2% per year
```

```
        elif self.age_years < 100:
            return 0.10  # 10% per year
        else:
            return 0.30  # 30% per year


# Константа — должна быть в constants.py
SECONDS_PER_GAME_YEAR: float = 365 * 24 * 3600  # 1 game-year = 365 game-days


class AgingEngine:
    """Ages NPCs based on elapsed game_time_seconds.

    PHYSICS LAYER — never called from kernel (tick-based) code.
    Called from physics overlay (reconcile_state, world_scheduler).
    """

    def advance_age(
        self,
        age_state: AgeState,
        elapsed_seconds: float,
    ) -> AgeState:
        """Advance NPC age by elapsed seconds.

        Args:
            age_state: current age state
            elapsed_seconds: game_time_seconds elapsed since last update

        Returns:
            New AgeState with updated age_years.
        """
        if elapsed_seconds <= 0:
            return age_state

        years_elapsed = elapsed_seconds / SECONDS_PER_GAME_YEAR
        new_age = age_state.age_years + years_elapsed

        return AgeState(
            age_years=new_age,
            birth_tick=age_state.birth_tick,
            birth_seconds=age_state.birth_seconds,
        )

    def should_die_of_old_age(
        self,
        age_state: AgeState,
        rng_seed: int,
    ) -> bool:
        """Check if NPC dies of old age this update.
```

```
    DETERMINISTIC: same (age_state, rng_seed) → same result.
    Uses KernelRNG pattern (but aging is physics, so seed from seconds).
    """
    import random
    rng = random.Random(rng_seed)
    prob = age_state.death_probability_per_year
    return rng.random() < prob

def get_age_modifiers(self, age_state: AgeState) -> Dict[str, float]:
    """Get gameplay modifiers based on age.

    Returns dict of modifiers:
        - max_hp_multiplier
        - willpower_multiplier
        - learning_rate_multiplier
        - social_authority_multiplier
    """
    stage = age_state.stage
    if stage == AgeStage.CHILD:
        return {
            "max_hp_multiplier": 0.5,
            "willpower_multiplier": 0.3,
            "learning_rate_multiplier": 2.0,  # children learn fast
            "social_authority_multiplier": 0.2,
        }
    elif stage == AgeStage.TEEN:
        return {
            "max_hp_multiplier": 0.8,
            "willpower_multiplier": 0.7,
            "learning_rate_multiplier": 1.5,
            "social_authority_multiplier": 0.5,
        }
    elif stage == AgeStage.ADULT:
        return {
            "max_hp_multiplier": 1.0,
            "willpower_multiplier": 1.0,
            "learning_rate_multiplier": 1.0,
            "social_authority_multiplier": 1.0,
        }
    elif stage == AgeStage.ELDER:
        return {
            "max_hp_multiplier": 0.7,
            "willpower_multiplier": 1.2,  # wisdom
            "learning_rate_multiplier": 0.6,
            "social_authority_multiplier": 1.5,  # respected
        }
    else:  # ANCIENT
        return {
            "max_hp_multiplier": 0.4,
```

```
    "willpower_multiplier": 1.0,
    "learning_rate_multiplier": 0.3,
    "social_authority_multiplier": 2.0,  # legend
}
```

4.4. Критерий приёмки

```
cd backend && python -c "
from app.services.world.aging import AgingEngine, AgeState, AgeStage

engine = AgingEngine()
age = AgeState(age_years=25.0)

# Advance 1 game-year
new_age = engine.advance_age(age, elapsed_seconds=365*24*3600)
assert abs(new_age.age_years - 26.0) < 0.01
assert new_age.stage == AgeStage.ADULT

# Test age stages
assert AgeState(age_years=10).stage == AgeStage.CHILD
assert AgeState(age_years=15).stage == AgeStage.TEEN
assert AgeState(age_years=30).stage == AgeStage.ADULT
assert AgeState(age_years=70).stage == AgeStage.ELDER
assert AgeState(age_years=85).stage == AgeStage.ANCIENT

# Test reproductive
assert AgeState(age_years=25).is_reproductive
assert not AgeState(age_years=15).is_reproductive
assert not AgeState(age_years=70).is_reproductive

print('AgingEngine OK')
"
```

---

5. STEP 4 — backend/app/services/world/birth_generator.py (NEW FILE)

5.1. Назначение

Spawn new NPC from parents. Использует Lineage для inheritance, WorldChronicle для логирования, KernelRNG для deterministic child generation.

5.2. Создать файл

Путь: backend/app/services/world/birth_generator.py

Содержимое:

```
"""
path: backend/app/services/world/birth_generator.py
Назначение: Spawn new NPC from parents. Uses Lineage, WorldChronicle,
```

```
KernelRNG.
Зависимости: app.services.world.lineage, app.services.world.world_chronicle,
             app.services.npc.kernel_rng
Основные сущности: BirthGenerator, BirthRequest, BirthResult

АРХИТЕКТУРНЫЙ ПРИНЦИП:
    Birth = deterministic function of (parents, tick, rng).
    Child inherits per Lineage.compute_inheritance rules.
    Birth event logged to WorldChronicle (objective truth).
    Child L1Chronicle starts empty (individual experience).
"""

from __future__ import annotations

import logging
from dataclasses import dataclass, field
from typing import Any, Dict, Optional, Tuple

from app.services.npc.kernel_rng import KernelRNG
from app.services.world.lineage import Lineage, InheritanceResult
from app.services.world.world_chronicle import WorldChronicle, WorldEvent,
WorldEventType
from app.services.world.aging import AgeState

logger = logging.getLogger(__name__)


@dataclass
class BirthRequest:
    """Request to generate a child."""
    parent_a_id: str
    parent_b_id: Optional[str]  # None for asexual reproduction
    tick: int
    game_time_seconds: float
    campaign_id: str
    child_name_hint: Optional[str] = None  # if None, generated


@dataclass
class BirthResult:
    """Result of birth generation."""
    child_id: str
    child_npc_dict: Dict[str, Any]  # full NPC dict for loader
    inheritance: InheritanceResult
    birth_event: WorldEvent


class BirthGenerator:
    """Generates new NPCs from parents.
```

```
    DETERMINISTIC: same (parents, tick, rng) → same child.
    """

    def __init__(
        self,
        chronicle: WorldChronicle,
        lineage: Lineage,
    ):
        self._chronicle = chronicle
        self._lineage = lineage

    def generate_child(
        self,
        request: BirthRequest,
        parent_a_dict: Dict[str, Any],
        parent_b_dict: Optional[Dict[str, Any]] = None,
    ) -> BirthResult:
        """Generate a new NPC from parents.

        Args:
            request: birth request with parent IDs and tick
            parent_a_dict: first parent's npc_dict
            parent_b_dict: second parent's npc_dict (or None for asexual)

        Returns:
            BirthResult with child NPC dict and event.
        """
        # Deterministic RNG for child generation
        # Seed = (tick, parent_a_id, parent_b_id) — unique per birth
        seed_key = f"{request.tick}:{request.parent_a_id}:{request.parent_b_id or
'asexual'}"
        rng = KernelRNG(tick=request.tick, npc_id=seed_key)

        # Compute inheritance
        inheritance = self._lineage.compute_inheritance(
            parent_npc_dict=parent_a_dict,
            tick=request.tick,
            rng=rng,
            second_parent_dict=parent_b_dict,
        )

        # Generate child ID
        child_id = self._generate_child_id(
            request.parent_a_id,
            request.parent_b_id,
            request.tick,
            rng,
        )
```

```
        # Generate child name
        child_name = request.child_name_hint or self._generate_child_name(
            parent_a_dict, parent_b_dict, rng
        )

        # Build child NPC dict
        child_dict = self._build_child_dict(
            child_id=child_id,
            child_name=child_name,
            inheritance=inheritance,
            request=request,
            parent_a_dict=parent_a_dict,
            parent_b_dict=parent_b_dict,
            rng=rng,
        )

        # Log birth event
        birth_event = WorldEvent.create(
            type=WorldEventType.BIRTH,
            tick=request.tick,
            actor_id=child_id,
            payload={
                "parents": [request.parent_a_id] + ([request.parent_b_id] if
request.parent_b_id else []),
                "child_name": child_name,
                "inherited_traits": inheritance.inherited_traits,
                "birth_game_time_seconds": request.game_time_seconds,
            },
            witness_ids=[request.parent_a_id] + ([request.parent_b_id] if
request.parent_b_id else []),
        )
        self._chronicle.log_event(birth_event)

        logger.info(
            f"[BIRTH] child={child_id} name={child_name} "
            f"parents=[{request.parent_a_id}, {request.parent_b_id or 'none'}] "
            f"tick={request.tick}"
        )

        return BirthResult(
            child_id=child_id,
            child_npc_dict=child_dict,
            inheritance=inheritance,
            birth_event=birth_event,
        )

    def _generate_child_id(
        self,
        parent_a_id: str,
        parent_b_id: Optional[str],
```

```
        tick: int,
        rng: KernelRNG,
    ) -> str:
        """Generate unique child ID."""
        # Format: {parent_a}_{parent_b_or_a}_child_{tick}_{random_suffix}
        parent_b_part = parent_b_id or parent_a_id
        # Use parent family name (part before _)
        family = parent_a_id.split("_")[0] if "_" in parent_a_id else parent_a_id
        suffix = rng.randint(1, 9999)
        return f"{family}_child_{tick}_{suffix}"

    def _generate_child_name(
        self,
        parent_a_dict: Dict[str, Any],
        parent_b_dict: Optional[Dict[str, Any]],
        rng: KernelRNG,
    ) -> str:
        """Generate child name (placeholder — should use proper name
generator)."""
        # TODO: integrate with proper name generator (cultural, linguistic)
        parent_a_name = parent_a_dict.get("name", "Unknown")
        # Simple placeholder
        return f"Child of {parent_a_name}"

    def _build_child_dict(
        self,
        child_id: str,
        child_name: str,
        inheritance: InheritanceResult,
        request: BirthRequest,
        parent_a_dict: Dict[str, Any],
        parent_b_dict: Optional[Dict[str, Any]],
        rng: KernelRNG,
    ) -> Dict[str, Any]:
        """Build full NPC dict for child (compatible with npc_loader)."""
        # Determine gender (50/50, deterministic)
        gender = "мужской" if rng.random() < 0.5 else "женский"

        # Determine archetype (inherit from dominant parent)
        archetype = parent_a_dict.get("_archetype", "commoner")

        # Birth tick — for age calculation
        birth_tick = request.tick
        birth_seconds = request.game_time_seconds

        child_dict = {
            "_version": "1.0",
            "_type": "individual",
            "_archetype": archetype,
            "_mixins": [],
```

```
            "id": child_id,
            "name": child_name,
            "name_forms": [child_name.lower()],  # TODO: proper name forms
            "tier": "minor",  # children start as minor
            "gender": gender,
            "description": f"Ребёнок {parent_a_dict.get('name', 'неизвестного')}",
            "voice_profile": parent_a_dict.get("voice_profile", ""),
            "backstory": f"Рождён от {parent_a_dict.get('name', 'неизвестного')}",
            "author_notes": "",
            "drives": inheritance.drives_base,
            "psyche": {
                "willpower": 30,  # children have lower willpower
                "breakpoint": 50,
                "loyalty_true": 0,
                "identity_integrity": 1.0,
                "pressure_resistance": 0.0,
                "stress": 0.0,
                "trauma_flags": list(inheritance.trauma_markers),
            },
            "abilities": parent_a_dict.get("abilities", {}),
            "flags": {
                "has_gold": False,
                "knows_secret": False,
                "is_enslaved": False,
                "planning_revenge": False,
                "is_dead": False,
            },
            "routine": {"schedule": {}},  # children have no fixed schedule
            "activity_map": {},
            "location": parent_a_dict.get("location", "tavern_silver_wolf"),
            "hp": 10,  # children have lower HP
            "max_hp": 10,
            "origin_events": [],
            # CK CORE LAYER fields
            "birth_tick": birth_tick,
            "birth_seconds": birth_seconds,
            "age_years": 0.0,
            "parent_ids": [request.parent_a_id] + ([request.parent_b_id] if
request.parent_b_id else []),
            "generation": inheritance.inherited_traits.get("generation", 1),
            "faction": inheritance.faction_membership,
        }

        return child_dict
```

5.3. Критерий приёмки

```
cd backend && python -c "
from app.services.world.birth_generator import BirthGenerator, BirthRequest
from app.services.world.lineage import Lineage
from app.services.world.world_chronicle import WorldChronicle
```

```
from app.services.npc.kernel_rng import KernelRNG

chronicle = WorldChronicle()
lineage = Lineage(chronicle)
generator = BirthGenerator(chronicle, lineage)

parent_a = {
    'id': 'maid_lusya',
    'name': 'Люся',
    '_archetype': 'maid',
    'drives': {'control': 0.3, 'significance': 0.2, 'fear': 0.4, 'desire': 0.1},
    'psyche': {'trauma_flags': ['abuse']},
    'location': 'tavern_silver_wolf',
}
parent_b = {
    'id': 'guard_borko',
    'name': 'Борко',
    '_archetype': 'guard',
    'drives': {'control': 0.4, 'significance': 0.3, 'fear': 0.1, 'desire': 0.2},
    'psyche': {'trauma_flags': []},
}

request = BirthRequest(
    parent_a_id='maid_lusya',
    parent_b_id='guard_borko',
    tick=1000,
    game_time_seconds=86400 * 365 * 5,  # 5 game-years
    campaign_id='test',
)

result = generator.generate_child(request, parent_a, parent_b)

assert result.child_id
assert result.child_npc_dict['id'] == result.child_id
assert result.child_npc_dict['parent_ids'] == ['maid_lusya', 'guard_borko']
assert result.child_npc_dict['generation'] == 1
assert result.child_npc_dict['age_years'] == 0.0

# Birth event logged
events = chronicle.query(type_filter=WorldEventType.BIRTH)
assert len(events) == 1

print('BirthGenerator OK')
"
```

6. STEP 5 — backend/app/services/world/succession_pipeline.py (NEW FILE)

6.1. Назначение

Death → heir inheritance pipeline. Когда NPC умирает, его heir наследует traits и занимает место.

6.2. Создать файл

Путь: backend/app/services/world/succession_pipeline.py

Содержимое:

```
"""
path: backend/app/services/world/succession_pipeline.py
Назначение: Death → heir inheritance pipeline.
Зависимости: app.services.world.lineage, app.services.world.world_chronicle,
             app.services.world.birth_generator, app.services.npc.kernel_rng
Основные сущности: SuccessionPipeline, SuccessionRequest, SuccessionResult

АРХИТЕКТУРНЫЙ ПРИНЦИП:
    Death = world event (objective truth in WorldChronicle).
    Heir = new kernel entity (NOT memory transfer).
    Continuity = lineage, NOT memory.

    L1Chronicle of deceased is NOT inherited.
    Heir starts with empty L1 (individual experience).

    Inherited:
        - drives_base (genetic, from deceased or their lineage)
        - trauma_markers (decayed probabilistic)
        - relationship_bias (social)
        - faction_membership
        - role/title (if deceased had one)
"""

from __future__ import annotations

import logging
from dataclasses import dataclass
from typing import Any, Dict, List, Optional

from app.services.npc.kernel_rng import KernelRNG
from app.services.world.lineage import Lineage, InheritanceResult
from app.services.world.world_chronicle import WorldChronicle, WorldEvent,
WorldEventType
from app.services.world.birth_generator import BirthGenerator, BirthRequest,
BirthResult

logger = logging.getLogger(__name__)
```

```python
@dataclass
class SuccessionRequest:
    """Request for succession after NPC death."""
    deceased_id: str
    death_tick: int
    death_seconds: float
    death_cause: str  # "old_age", "killed_by:NPC_ID", "disease", etc.
    campaign_id: str
    deceased_dict: Dict[str, Any]  # full NPC dict of deceased


@dataclass
class SuccessionResult:
    """Result of succession."""
    heir_id: Optional[str]  # None if no heir available
    heir_npc_dict: Optional[Dict[str, Any]]
    inheritance: Optional[InheritanceResult]
    death_event: WorldEvent
    succession_event: Optional[WorldEvent]  # None if no heir


class SuccessionPipeline:
    """Handles death → heir inheritance.

    PIPELINE:
        1. Log DEATH event to WorldChronicle.
        2. Find heir (oldest child, or designated, or none).
        3. If heir found:
            a. Generate heir NPC (via BirthGenerator with deceased as "parent").
            b. Apply inheritance (drives, traumas, relationships, faction, role).
            c. Log SUCCESSION event.
        4. If no heir:
            - Lineage ends.
            - Role may become vacant (faction-level consequences).
    """

    def __init__(
        self,
        chronicle: WorldChronicle,
        lineage: Lineage,
        birth_generator: BirthGenerator,
    ):
        self._chronicle = chronicle
        self._lineage = lineage
        self._birth_generator = birth_generator

    def process_death(self, request: SuccessionRequest) -> SuccessionResult:
        """Process NPC death and trigger succession if applicable."""
```

```
# 1. Log DEATH event
death_event = WorldEvent.create(
    type=WorldEventType.DEATH,
    tick=request.death_tick,
    actor_id=request.deceased_id,
    payload={
        "cause": request.death_cause,
        "death_seconds": request.death_seconds,
        "deceased_name": request.deceased_dict.get("name", "unknown"),
        "generation": request.deceased_dict.get("generation", 0),
    },
    witness_ids=[],  # TODO: compute witnesses from scene
)
self._chronicle.log_event(death_event)

logger.info(
    f"[DEATH] npc={request.deceased_id} "
    f"cause={request.death_cause} tick={request.death_tick}"
)

# 2. Find heir
heir_id = self._find_heir(request.deceased_id, request.deceased_dict)

if heir_id is None:
    # No heir — lineage ends
    logger.info(
        f"[SUCCESSION] no heir for {request.deceased_id}, lineage ends"
    )
    return SuccessionResult(
        heir_id=None,
        heir_npc_dict=None,
        inheritance=None,
        death_event=death_event,
        succession_event=None,
    )

# 3. Generate heir (as if "born" from deceased — symbolic inheritance)
birth_request = BirthRequest(
    parent_a_id=request.deceased_id,
    parent_b_id=None,  # succession is "asexual" inheritance
    tick=request.death_tick,
    game_time_seconds=request.death_seconds,
    campaign_id=request.campaign_id,
    child_name_hint=self._generate_heir_name(request.deceased_dict),
)

# Deceased is "parent" for inheritance purposes
# Heir is NOT biological child — they inherit social position
birth_result = self._birth_generator.generate_child(
```

```
            request=birth_request,
            parent_a_dict=request.deceased_dict,
            parent_b_dict=None,
        )

        # Override heir-specific fields
        heir_dict = birth_result.child_npc_dict
        heir_dict["tier"] = request.deceased_dict.get("tier", "minor")  # inherit tier
        heir_dict["age_years"] = self._compute_heir_age(
            request.deceased_dict, request.death_seconds
        )
        heir_dict["hp"] = request.deceased_dict.get("hp", 10)  # inherit HP
        heir_dict["max_hp"] = request.deceased_dict.get("max_hp", 10)
        heir_dict["location"] = request.deceased_dict.get("location",
"tavern_silver_wolf")
        # Inherit routine (social position)
        heir_dict["routine"] = request.deceased_dict.get("routine", {})
        heir_dict["activity_map"] = request.deceased_dict.get("activity_map", {})

        # 4. Log SUCCESSION event
        succession_event = WorldEvent.create(
            type=WorldEventType.SUCCESSION,
            tick=request.death_tick,
            actor_id=heir_id,
            target_id=request.deceased_id,
            payload={
                "deceased_id": request.deceased_id,
                "deceased_name": request.deceased_dict.get("name", "unknown"),
                "heir_name": heir_dict.get("name", "unknown"),
                "inherited_role": request.deceased_dict.get("tier", "minor"),
                "inherited_faction": heir_dict.get("faction"),
            },
            witness_ids=[],
        )
        self._chronicle.log_event(succession_event)

        logger.info(
            f"[SUCCESSION] heir={heir_id} "
            f"succeeds={request.deceased_id} tick={request.death_tick}"
        )

        return SuccessionResult(
            heir_id=heir_id,
            heir_npc_dict=heir_dict,
            inheritance=birth_result.inheritance,
            death_event=death_event,
            succession_event=succession_event,
        )

    def _find_heir(
```

```
        self,
        deceased_id: str,
        deceased_dict: Dict[str, Any],
    ) -> Optional[str]:
        """Find heir for deceased.

        Priority:
            1. Oldest living child (from Lineage)
            2. Designated heir (from deceased_dict.get("designated_heir"))
            3. None (lineage ends)
        """
        # Check Lineage for children
        record = self._lineage.get_record(deceased_id)
        if record and record.children_ids:
            # Find oldest living child
            for child_id in record.children_ids:
                child_record = self._lineage.get_record(child_id)
                if child_record and child_record.is_alive:
                    return child_id

        # Check designated heir
        designated = deceased_dict.get("designated_heir")
        if designated:
            return designated

        return None

    def _generate_heir_name(self, deceased_dict: Dict[str, Any]) -> str:
        """Generate heir name (placeholder)."""
        deceased_name = deceased_dict.get("name", "Unknown")
        return f"Heir of {deceased_name}"

    def _compute_heir_age(
        self,
        deceased_dict: Dict[str, Any],
        death_seconds: float,
    ) -> float:
        """Compute heir age.

        Heir is assumed to be adult (18+) for gameplay reasons.
        If heir is biological child, their age is computed from their birth.
        For now, default to 18 (legal adult).
        """
        return 18.0  # TODO: compute from actual child birth_seconds
```

6.3. Критерий приёмки

```
cd backend && python -c "
from app.services.world.succession_pipeline import SuccessionPipeline,
SuccessionRequest
from app.services.world.lineage import Lineage
```

```
from app.services.world.world_chronicle import WorldChronicle
from app.services.world.birth_generator import BirthGenerator

chronicle = WorldChronicle()
lineage = Lineage(chronicle)
birth_gen = BirthGenerator(chronicle, lineage)
pipeline = SuccessionPipeline(chronicle, lineage, birth_gen)

deceased = {
    'id': 'tavern_keeper_tornin',
    'name': 'Торнин',
    'tier': 'major',
    'drives': {'control': 0.4, 'significance': 0.3, 'fear': 0.1, 'desire': 0.2},
    'location': 'tavern_silver_wolf',
    'hp': 30,
    'max_hp': 30,
}

request = SuccessionRequest(
    deceased_id='tavern_keeper_tornin',
    death_tick=2000,
    death_seconds=86400 * 365 * 50,
    death_cause='old_age',
    campaign_id='test',
    deceased_dict=deceased,
)

result = pipeline.process_death(request)

# Death event logged
assert result.death_event.type.value == 'death'

# If heir found, succession event logged
if result.heir_id:
    assert result.succession_event is not None
    assert result.succession_event.type.value == 'succession'
    assert result.heir_npc_dict is not None
    print(f'Heir generated: {result.heir_id}')
else:
    print('No heir (lineage ends)')

print('SuccessionPipeline OK')
"
```

---

7. STEP 6 — backend/app/services/world/cross_npc_relationships.py (NEW FILE)

7.1. Назначение

Dynamic NPC→NPC relationships (не только player→NPC). NPC помнят, кто их предал, кто спас, кто родственник.

7.2. Создать файл

Путь: backend/app/services/world/cross_npc_relationships.py

Содержимое:

```
"""
path: backend/app/services/world/cross_npc_relationships.py
Назначение: Dynamic NPC→NPC relationships (not just player→NPC).
Зависимости: app.services.world.world_chronicle
Основные сущности: CrossNPCRelationships, RelationshipEntry

АРХИТЕКТУРНЫЙ ПРИНЦИП:
    Existing: RelationshipStore (player→NPC only)
    NEW: CrossNPCRelationships (NPC→NPC dynamic)

    NOT replace — ADD. Existing player→NPC relationships unchanged.

PERSISTENCE:
    SQLite (durable). Tick-indexed for queries.
"""

from __future__ import annotations

import logging
from dataclasses import dataclass, field
from typing import Dict, List, Optional, Tuple

logger = logging.getLogger(__name__)


@dataclass
class RelationshipEntry:
    """NPC→NPC relationship state."""
    source_id: str  # NPC who has the feeling
    target_id: str  # NPC toward whom
    trust: float = 0.0  # -100..100
    fear: float = 0.0  # 0..100
    affection: float = 0.0  # -100..100
    debt: float = 0.0  # -100..100 (positive = source owes target)
    last_update_tick: int = 0
    history: List[Tuple[int, str, float]] = field(default_factory=list)  # (tick, event,
```

```
delta)

    def apply_delta(
        self,
        trust_delta: float = 0.0,
        fear_delta: float = 0.0,
        affection_delta: float = 0.0,
        debt_delta: float = 0.0,
        tick: int = 0,
        event: str = "",
    ) -> None:
        """Apply relationship delta with clamping and history logging."""
        self.trust = max(-100.0, min(100.0, self.trust + trust_delta))
        self.fear = max(0.0, min(100.0, self.fear + fear_delta))
        self.affection = max(-100.0, min(100.0, self.affection + affection_delta))
        self.debt = max(-100.0, min(100.0, self.debt + debt_delta))
        self.last_update_tick = tick
        if event:
            self.history.append((tick, event, trust_delta + fear_delta + affection_delta))


class CrossNPCRelationships:
    """Dynamic NPC→NPC relationships.

    Persistence: in-memory cache + SQLite (TODO).
    Key: (source_id, target_id) — directional.
    """

    def __init__(self):
        self._relationships: Dict[Tuple[str, str], RelationshipEntry] = {}

    def get(self, source_id: str, target_id: str) -> RelationshipEntry:
        """Get or create relationship entry."""
        key = (source_id, target_id)
        if key not in self._relationships:
            self._relationships[key] = RelationshipEntry(
                source_id=source_id,
                target_id=target_id,
            )
        return self._relationships[key]

    def apply_event(
        self,
        source_id: str,
        target_id: str,
        event_type: str,
        tick: int,
        trust_delta: float = 0.0,
        fear_delta: float = 0.0,
        affection_delta: float = 0.0,
```

```
    debt_delta: float = 0.0,
) -> None:
    """Apply relationship change from world event.

    Event types and their deltas:
        - 'saved_life': trust +30, affection +20
        - 'betrayed': trust -50, affection -30
        - 'killed_ally': trust -80, fear +40
        - 'gift_given': trust +10, affection +5, debt +5
        - 'insulted': trust -10, affection -10
        - 'helped': trust +15, affection +10
    """
    entry = self.get(source_id, target_id)
    entry.apply_delta(
        trust_delta=trust_delta,
        fear_delta=fear_delta,
        affection_delta=affection_delta,
        debt_delta=debt_delta,
        tick=tick,
        event=event_type,
    )

def get_all_for(self, npc_id: str) -> Dict[str, RelationshipEntry]:
    """Get all relationships where npc_id is source."""
    return {
        target_id: entry
        for (source, target_id), entry in self._relationships.items()
        if source == npc_id
    }

def get_all_toward(self, npc_id: str) -> Dict[str, RelationshipEntry]:
    """Get all relationships where npc_id is target."""
    return {
        source_id: entry
        for (source_id, target), entry in self._relationships.items()
        if target == npc_id
    }

def get_family_bias(
    self,
    npc_id: str,
    lineage_ancestors: List[str],
) -> Dict[str, Tuple[float, float]]:
    """Get inherited relationship bias from ancestors.

    If NPC's ancestor had relationship with target_id,
    NPC inherits partial bias.
    """
    bias = {}
    for ancestor_id in lineage_ancestors:
```

```
        ancestor_rels = self.get_all_for(ancestor_id)
        for target_id, entry in ancestor_rels.items():
            if target_id == npc_id:
                continue  # don't inherit feelings about self
            # Inherit 30% of ancestor's feelings
            inherited_trust = entry.trust * 0.3
            inherited_fear = entry.fear * 0.3
            if target_id in bias:
                # Average with existing
                bias[target_id] = (
                    (bias[target_id][0] + inherited_trust) / 2,
                    (bias[target_id][1] + inherited_fear) / 2,
                )
            else:
                bias[target_id] = (inherited_trust, inherited_fear)
    return bias
```

7.3. Критерий приёмки

```
cd backend && python -c "
from app.services.world.cross_npc_relationships import CrossNPCRelationships

rels = CrossNPCRelationships()

# NPC saves another NPC
rels.apply_event('maid_lusya', 'guard_borko', 'saved_life', tick=100, trust_delta=30,
affection_delta=20)

entry = rels.get('maid_lusya', 'guard_borko')
assert entry.trust == 30
assert entry.affection == 20
assert len(entry.history) == 1

# NPC betrays
rels.apply_event('thief_shadow', 'maid_lusya', 'betrayed', tick=200,
trust_delta=-50, affection_delta=-30)
entry2 = rels.get('thief_shadow', 'maid_lusya')
assert entry2.trust == -50

# Get all relationships for NPC
lusya_rels = rels.get_all_for('maid_lusya')
assert 'guard_borko' in lusya_rels

print('CrossNPCRelationships OK')
"
```

---

8. STEP 7 — backend/app/services/events/event_types.py (MODIFY)

8.1. Назначение

Добавить CK event types в EventType enum.

8.2. Изменение

ЗАМЕНИ (в enum EventType, после существующих world events):

```
    # — Мир ———————————————————————————————————————————
    TIME_PASSED      = "time_passed"
    WEATHER_CHANGED  = "weather_changed"
    FACTION_EVENT    = "faction_event"
    WORLD_TICK       = "world_tick"       # проактивный тик мира (Фаза 3.4)
```

НА:

```
    # — Мир ———————————————————————————————————————————
    TIME_PASSED      = "time_passed"
    WEATHER_CHANGED  = "weather_changed"
    FACTION_EVENT    = "faction_event"
    WORLD_TICK       = "world_tick"       # проактивный тик мира (Фаза 3.4)

    # — CK CORE LAYER: NPC lifecycle ——————————————————
    NPC_BIRTH        = "npc_birth"        # NPC родился
    NPC_DEATH        = "npc_death"        # NPC умер
    NPC_MARRIAGE     = "npc_marriage"     # NPC вступил в брак
    NPC_SUCCESSION   = "npc_succession"   # NPC занял место покойного
    NPC_INHERITANCE  = "npc_inheritance"  # NPC унаследовал traits
```

8.3. Критерий приёмки

```
cd backend && python -c "
from app.services.events.event_types import EventType
assert EventType.NPC_BIRTH.value == 'npc_birth'
assert EventType.NPC_DEATH.value == 'npc_death'
assert EventType.NPC_SUCCESSION.value == 'npc_succession'
print('EventType CK additions OK')
"
```

---

9. STEP 8 — backend/app/models/npc_state.py (MODIFY)

9.1. Назначение

Добавить CK fields в NPCState: lineage, age, parent_ids, generation.

9.2. Изменение

НАЙТИ в NPCState dataclass (после trauma_markers поля, примерно строка 590):

ДОБАВИТЬ поля:

```
    # ═══ CK CORE LAYER: Lineage & Aging ═══════════════════════════════
    # ADR-CK-001: NPC genealogy and aging for multi-generational play.
    # Aging = physics layer (seconds-based, NOT tick-based).
    # Lineage = kernel layer (tick-indexed, deterministic).

    # Lineage (kernel layer, tick-indexed)
    parent_ids: Tuple[str, ...] = ()  # 0 (founder), 1 (asexual), 2 (sexual)
    children_ids: Tuple[str, ...] = ()
    generation: int = 0  # 0 = founder, 1 = first generation, etc.
    birth_tick: int = 0  # when NPC was born (causal tick)

    # Aging (physics layer, seconds-based)
    age_years: float = 25.0  # default adult
    birth_seconds: float = 0.0  # game_time_seconds at birth

    # Succession
    designated_heir: Optional[str] = None  # explicit heir override
```

9.3. Обновить write_to_legacy (serialization)

НАЙТИ в write_to_legacy (примерно строка 700):

ДОБАВИТЬ перед # Физическое состояние:

```
        # CK CORE LAYER: Lineage & Aging
        if state.parent_ids:
            npc_dict["parent_ids"] = list(state.parent_ids)
        if state.children_ids:
            npc_dict["children_ids"] = list(state.children_ids)
        npc_dict["generation"] = state.generation
        npc_dict["birth_tick"] = state.birth_tick
        npc_dict["age_years"] = state.age_years
        npc_dict["birth_seconds"] = state.birth_seconds
        if state.designated_heir:
            npc_dict["designated_heir"] = state.designated_heir
```

9.4. Обновить from_legacy (deserialization)

НАЙТИ в from_legacy (примерно строка 870):

ДОБАВИТЬ в NPCState(...) constructor:

```
            # CK CORE LAYER: Lineage & Aging
            parent_ids        = tuple(npc_dict.get("parent_ids", [])),
```

```
        children_ids      = tuple(npc_dict.get("children_ids", [])),
        generation        = int(npc_dict.get("generation", 0)),
        birth_tick        = int(npc_dict.get("birth_tick", 0)),
        age_years         = float(npc_dict.get("age_years", 25.0)),
        birth_seconds     = float(npc_dict.get("birth_seconds", 0.0)),
        designated_heir   = npc_dict.get("designated_heir"),
```

9.5. Критерий приёмки

```
cd backend && python -c "
from app.models.npc_state import NPCState
state = NPCState(npc_id='test')
assert state.parent_ids == ()
assert state.generation == 0
assert state.age_years == 25.0
assert state.designated_heir is None

# Test with values
state2 = NPCState(
    npc_id='child_1',
    parent_ids=('maid_lusya', 'guard_borko'),
    generation=1,
    age_years=5.0,
)
assert state2.parent_ids == ('maid_lusya', 'guard_borko')
assert state2.generation == 1
assert state2.age_years == 5.0
print('NPCState CK fields OK')
"
```

---

10. STEP 9 — backend/app/services/npc/npc_loader.py (MODIFY)

10.1. Назначение

Загружать CK fields из config JSON.

10.2. Изменение

НАЙТИ в load_npcs_merged или эквивалентной функции, где читаются fields из JSON:

ДОБАВИТЬ чтение CK fields:

```
        # CK CORE LAYER: Lineage & Aging
        "parent_ids": npc_dict.get("parent_ids", []),
        "children_ids": npc_dict.get("children_ids", []),
        "generation": npc_dict.get("generation", 0),
        "birth_tick": npc_dict.get("birth_tick", 0),
        "age_years": npc_dict.get("age_years", 25.0),
```

```
        "birth_seconds": npc_dict.get("birth_seconds", 0.0),
        "designated_heir": npc_dict.get("designated_heir"),
```

10.3. Критерий приёмки

Существующие NPC configs (lusya.json, etc.) загружаются без ошибок (CK fields default).

---

11. STEP 10 — backend/app/services/tick_orchestrator.py (MODIFY)

11.1. Назначение

Интегрировать CK services в tick pipeline: aging update, succession check.

11.2. Изменение 1 — инициализация CK services

НАЙТИ __init__ TickOrchestrator (примерно строка 280-300):

ДОБАВИТЬ инициализацию:

```
        # CK CORE LAYER services
        from app.services.world.world_chronicle import WorldChronicle
        from app.services.world.lineage import Lineage
        from app.services.world.aging import AgingEngine
        from app.services.world.birth_generator import BirthGenerator
        from app.services.world.succession_pipeline import SuccessionPipeline
        from app.services.world.cross_npc_relationships import
CrossNPCRelationships

        # Эти сервисы создаются per-campaign, не per-orchestrator.
        # В реальном коде — injecting через factory или lazy init.
        self._ck_services: Dict[str, dict] = {}  # campaign_id → services

    def _get_ck_services(self, campaign_id: str) -> dict:
        """Get or create CK services for campaign."""
        if campaign_id not in self._ck_services:
            from app.services.memory.sqlite_store import SQLiteStore
            store = SQLiteStore.default()  # или как создаётся

            chronicle = WorldChronicle(store=store, campaign_id=campaign_id)
            lineage = Lineage(chronicle)
            birth_gen = BirthGenerator(chronicle, lineage)
            succession = SuccessionPipeline(chronicle, lineage, birth_gen)
            aging = AgingEngine()
            cross_rels = CrossNPCRelationships()

            self._ck_services[campaign_id] = {
                "chronicle": chronicle,
                "lineage": lineage,
```

```
            "birth_generator": birth_gen,
            "succession": succession,
            "aging": aging,
            "cross_relationships": cross_rels,
        }
    return self._ck_services[campaign_id]
```

11.3. Изменение 2 — фаза aging (physics layer)

ДОБАВИТЬ новый метод:

```
def _phase_physics_aging(self, ctx: _TickContext) -> None:
    """PHYSICS LAYER: advance NPC ages based on elapsed seconds.

    NOT in causal tick pipeline. Called from physics overlay
    (after game_time_seconds updated).
    """
    ck = self._get_ck_services(ctx.campaign_id)
    aging = ck["aging"]

    # Compute elapsed seconds since last aging update
    current_seconds = ctx.scene_state.get("game_time_seconds", 0.0)
    last_aging_key = f"_last_aging_seconds_{ctx.campaign_id}"
    last_seconds = getattr(self, last_aging_key, current_seconds)
    elapsed = current_seconds - last_seconds

    if elapsed <= 0:
        return

    # Age all NPCs
    for npc_dict in ctx.all_npcs_raw:
        npc_id = npc_dict.get("id", npc_dict.get("npc_id"))
        if not npc_id:
            continue

        # Skip dead NPCs
        if npc_dict.get("psyche", {}).get("state") == "broken" and \
           npc_dict.get("body_state", {}).get("life_status") == "DEAD":
            continue

        # Update age
        current_age = npc_dict.get("age_years", 25.0)
        new_age = current_age + (elapsed / (365 * 24 * 3600))  # seconds to years
        npc_dict["age_years"] = new_age

        # Check natural death
        from app.services.world.aging import AgeState
        age_state = AgeState(
            age_years=new_age,
            birth_tick=npc_dict.get("birth_tick", 0),
            birth_seconds=npc_dict.get("birth_seconds", 0.0),
```

```
            )

            # Deterministic death check (seed from seconds, not tick)
            import random
            death_rng = random.Random(int(current_seconds) + hash(npc_id))
            if aging.should_die_of_old_age(age_state, death_rng.seed):
                # Trigger succession
                self._trigger_succession(ctx, npc_id, npc_dict, "old_age")

        setattr(self, last_aging_key, current_seconds)

    def _trigger_succession(
        self,
        ctx: _TickContext,
        deceased_id: str,
        deceased_dict: dict,
        cause: str,
    ) -> None:
        """Trigger succession pipeline for deceased NPC."""
        ck = self._get_ck_services(ctx.campaign_id)
        from app.services.world.succession_pipeline import SuccessionRequest

        request = SuccessionRequest(
            deceased_id=deceased_id,
            death_tick=ctx.tick_number,
            death_seconds=ctx.scene_state.get("game_time_seconds", 0.0),
            death_cause=cause,
            campaign_id=ctx.campaign_id,
            deceased_dict=deceased_dict,
        )

        result = ck["succession"].process_death(request)

        if result.heir_npc_dict:
            # Add heir to NPC list (spawn)
            ctx.all_npcs_raw.append(result.heir_npc_dict)
            ctx.dirty_npcs.add(result.heir_id)
            logger.info(f"[CK_SUCCESSION] heir spawned: {result.heir_id}")
```

11.4. Изменение 3 — вызов aging из execute

НАЙТИ в execute или execute_player_finalize, после обновления game_time_seconds:

ДОБАВИТЬ вызов:

```
        # PHYSICS LAYER: aging (after game_time_seconds updated)
        self._phase_physics_aging(ctx)
```

11.5. Критерий приёмки

```
cd backend && python -c "
from app.services.tick_orchestrator import TickOrchestrator
# Проверить, что _get_ck_services существует
import inspect
assert hasattr(TickOrchestrator, '_get_ck_services')
assert hasattr(TickOrchestrator, '_phase_physics_aging')
assert hasattr(TickOrchestrator, '_trigger_succession')
print('TickOrchestrator CK integration OK')
"
```

---

12. STEP 11 — backend/tests/test_ck_core_layer.py (NEW FILE)

12.1. Назначение

Integration tests для CK CORE LAYER.

12.2. Создать файл

Путь: backend/tests/test_ck_core_layer.py

Содержимое:

```python
"""Integration tests for CK CORE LAYER."""
import pytest
from app.services.world.world_chronicle import WorldChronicle, WorldEvent, WorldEventType
from app.services.world.lineage import Lineage
from app.services.world.aging import AgingEngine, AgeState, AgeStage
from app.services.world.birth_generator import BirthGenerator, BirthRequest
from app.services.world.succession_pipeline import SuccessionPipeline, SuccessionRequest
from app.services.world.cross_npc_relationships import CrossNPCRelationships
from app.services.npc.kernel_rng import KernelRNG


class TestWorldChronicle:
    def test_log_and_query(self, tmp_path):
        """Event логируется и находится."""
        from app.services.memory.sqlite_store import SQLiteStore
        store = SQLiteStore(str(tmp_path / "test.db"))
        chronicle = WorldChronicle(store=store, campaign_id="test")

        event = WorldEvent.create(
            type=WorldEventType.BIRTH,
            tick=100,
            actor_id="child_1",
            payload={"parents": ["maid_lusya", "guard_borko"]},
        )
```

```
        chronicle.log_event(event)

        # Simulate restart
        chronicle2 = WorldChronicle(store=store, campaign_id="test")
        events = chronicle2.query(tick_from=0)
        assert len(events) == 1
        assert events[0].actor_id == "child_1"


class TestLineage:
    def test_inheritance_deterministic(self):
        """Same (parent, tick, rng) → same inheritance."""
        chronicle = WorldChronicle()
        lineage = Lineage(chronicle)

        parent = {
            'id': 'maid_lusya',
            'drives': {'control': 0.3, 'significance': 0.2, 'fear': 0.4, 'desire': 0.1},
            'psyche': {'trauma_flags': ['abuse']},
        }

        rng1 = KernelRNG(tick=1000, npc_id='child_1')
        rng2 = KernelRNG(tick=1000, npc_id='child_1')

        result1 = lineage.compute_inheritance(parent, 1000, rng1)
        result2 = lineage.compute_inheritance(parent, 1000, rng2)

        assert result1.drives_base == result2.drives_base
        assert result1.trauma_markers == result2.trauma_markers

    def test_drives_normalization(self):
        """Inherited drives sum to 1.0 (Закон Сохранения Я)."""
        chronicle = WorldChronicle()
        lineage = Lineage(chronicle)

        parent = {
            'id': 'parent',
            'drives': {'control': 0.3, 'significance': 0.2, 'fear': 0.4, 'desire': 0.1},
        }
        rng = KernelRNG(tick=100, npc_id='child')

        result = lineage.compute_inheritance(parent, 100, rng)
        total = sum(result.drives_base.values())
        assert abs(total - 1.0) < 0.01


class TestAging:
    def test_age_advances_with_seconds(self):
        """Age increases with elapsed seconds, not ticks."""
        engine = AgingEngine()
```

```
        age = AgeState(age_years=25.0)

        # 1 game-year elapsed
        new_age = engine.advance_age(age, elapsed_seconds=365 * 24 * 3600)
        assert abs(new_age.age_years - 26.0) < 0.01

    def test_age_stages(self):
        assert AgeState(age_years=10).stage == AgeStage.CHILD
        assert AgeState(age_years=15).stage == AgeStage.TEEN
        assert AgeState(age_years=30).stage == AgeStage.ADULT
        assert AgeState(age_years=70).stage == AgeStage.ELDER
        assert AgeState(age_years=85).stage == AgeStage.ANCIENT

    def test_reproductive_age(self):
        assert AgeState(age_years=25).is_reproductive
        assert not AgeState(age_years=15).is_reproductive
        assert not AgeState(age_years=70).is_reproductive


class TestBirthGenerator:
    def test_generate_child_deterministic(self):
        """Same (parents, tick) → same child."""
        chronicle = WorldChronicle()
        lineage = Lineage(chronicle)
        generator = BirthGenerator(chronicle, lineage)

        parent_a = {'id': 'maid_lusya', 'name': 'Люся', 'drives': {'control': 0.3,
'significance': 0.2, 'fear': 0.4, 'desire': 0.1}}
        parent_b = {'id': 'guard_borko', 'name': 'Борко', 'drives': {'control': 0.4,
'significance': 0.3, 'fear': 0.1, 'desire': 0.2}}

        request = BirthRequest(
            parent_a_id='maid_lusya',
            parent_b_id='guard_borko',
            tick=1000,
            game_time_seconds=0.0,
            campaign_id='test',
        )

        result1 = generator.generate_child(request, parent_a, parent_b)
        result2 = generator.generate_child(request, parent_a, parent_b)

        assert result1.child_id == result2.child_id
        assert result1.child_npc_dict['drives'] == result2.child_npc_dict['drives']


class TestSuccessionPipeline:
    def test_death_logs_event(self, tmp_path):
        """Death logs DEATH event to WorldChronicle."""
        from app.services.memory.sqlite_store import SQLiteStore
```

```
        store = SQLiteStore(str(tmp_path / "test.db"))
        chronicle = WorldChronicle(store=store, campaign_id="test")
        lineage = Lineage(chronicle)
        birth_gen = BirthGenerator(chronicle, lineage)
        pipeline = SuccessionPipeline(chronicle, lineage, birth_gen)

        deceased = {'id': 'npc_1', 'name': 'Test', 'drives': {'control': 0.25,
'significance': 0.25, 'fear': 0.25, 'desire': 0.25}}
        request = SuccessionRequest(
            deceased_id='npc_1',
            death_tick=100,
            death_seconds=0.0,
            death_cause='old_age',
            campaign_id='test',
            deceased_dict=deceased,
        )

        result = pipeline.process_death(request)

        assert result.death_event.type == WorldEventType.DEATH
        # Heir may or may not exist (depends on lineage state)

        # Verify event persisted
        chronicle2 = WorldChronicle(store=store, campaign_id="test")
        events = chronicle2.query(type_filter=WorldEventType.DEATH)
        assert len(events) == 1


class TestCrossNPCRelationships:
    def test_relationship_persists(self):
        """NPC→NPC relationship persists."""
        rels = CrossNPCRelationships()
        rels.apply_event('maid_lusya', 'guard_borko', 'saved_life', tick=100,
trust_delta=30)

        entry = rels.get('maid_lusya', 'guard_borko')
        assert entry.trust == 30

        # Bidirectional check
        reverse = rels.get('guard_borko', 'maid_lusya')
        assert reverse.trust == 0  # not set
```

12.3. Критерий приёмки

```
cd backend && python -m pytest tests/test_ck_core_layer.py -v
```

Все тесты должны проходить.

---

13. СВОДКА ВНЕДРЕНИЯ

Step	Файл	Тип	Что делает	Время
1	world_chronicle.py	NEW	Global event log (objective truth)	1.5 часа
2	lineage.py	NEW	Parent→child inheritance graph	1 час
3	aging.py	NEW	Seconds-based aging (physics layer)	45 мин
4	birth_generator.py	NEW	Spawn NPC from parents	1 час
5	succession_pipeline.py	NEW	Death → heir inheritance	1 час
6	cross_npc_relationships.py	NEW	NPC→NPC dynamic relationships	45 мин
7	event_types.py	MODIFY	Add CK event types	15 мин
8	npc_state.py	MODIFY	Add CK fields (lineage, age)	45 мин
9	npc_loader.py	MODIFY	Load CK fields from config	30 мин
10	tick_orchestrator.py	MODIFY	Integrate CK services	1.5 часа
11	test_ck_core_layer.py	NEW	Integration tests	1 час

Итого: ~10 часов работы = 1.5 дня.

---

14. ЧТО CK CORE LAYER ГАРАНТИРУЕТ

После применения:

✅ NPC genealogy (parent→child graph, ancestors/descendants) ✅ NPC aging (seconds-based, physics layer) ✅ Birth events (deterministic child generation from parents) ✅ Death → succession (heir inherits traits, role) ✅ WorldChronicle (cross-NPC, cross-generation event log) ✅ Cross-NPC

relationships (dynamic, not just player→NPC) ✅ Multi-generational play foundation

Что НЕ решается (намеренно):

- Marriage system (кто с кем женится) — нужен отдельный ADR
- Faction dynamics (войны, альянсы) — нужен отдельный ADR
- Player lineage (династия игрока) — нужен отдельный ADR
- Settlement/city dynamics — future

Архитектурная истина:

> CK CORE LAYER — additive systems поверх deterministic kernel. Не переписывает существующий код. Kernel isolation (из Kernel Isolation Repair v0.1) сохранён.

---

15. СЛЕДУЮЩИЙ ШАГ (после применения CK CORE LAYER)

После подтверждения, что CK CORE LAYER применён и тесты проходят, следующий слой:

«CK GAMEPLAY LAYER»

- Player lineage (династия игрока, respawn as heir)
- Marriage system (кто с кем женится, политические браки)
- Faction dynamics (войны, альянсы, дипломатия)
- Settlement management (города, провинции)
- Death screen → heir selection UI
- Calendar UI (generation view, family tree)

Это уже gameplay layer, не simulation layer.

---

16. РОЛЛБЭК СТРАТЕГИЯ

Если CK CORE LAYER ломает существующие тесты:

1. **Откатить STEP 10 (tick_orchestrator)** — убрать CK service integration
2. **Откатить STEP 8-9 (npc_state, npc_loader)** — убрать CK fields
3. **Откатить STEP 7 (event_types)** — убрать CK event types
4. **Откатить STEP 1-6** — удалить новые файлы

Каждый STEP — independently testable. Откатывать в обратном порядке.

---

17. ПРИНЦИПЫ, КОТОРЫЕ CK CORE LAYER СОБЛЮДАЕТ

1. **Additive only** — не меняем существующий DecisionHub, StateApplicator, LifeEngine
2. **Kernel isolation preserved** — все CK events tick-indexed, deterministic
3. **Physics layer respected** — aging = seconds-based, not tick-based
4. **L1 vs WorldChronicle separation** — subjective vs objective, не пересекаются
5. **Inheritance rules non-negotiable** — что наследуется, что нет (зафиксировано)
6. **Backward compatibility** — существующие NPC configs работают (CK fields default)

---

18. ФИНАЛЬНАЯ ПРОВЕРКА

```
# 1. WorldChronicle tests
cd backend && python -m pytest tests/test_world_chronicle.py -v

# 2. CK core layer tests
cd backend && python -m pytest tests/test_ck_core_layer.py -v

# 3. Existing tests still pass
cd backend && python -m pytest tests/ -v --timeout=60

# 4. Kernel isolation still valid
cd backend && python -m tests.sandbox.SUPERBOX.kernel_isolation_test
# Ожидание: 0% causal drift (не изменилось)

# 5. NPC configs load with CK fields
cd backend && python -c "
from app.services.npc.npc_loader import load_npcs_merged
npcs = load_npcs_merged()
for npc in npcs:
    assert 'age_years' in npc or 'age_years' not in npc  # default OK
print(f'Loaded {len(npcs)} NPCs with CK fields')
"
```

Если все 5 проверок проходят — CK CORE LAYER применён успешно.

КОНЕЦ ИНСТРУКЦИИ

Документ содержит: - 6 новых файлов (CK services) - 4 modify (event_types, npc_state, npc_loader, tick_orchestrator) - 1 test file (integration tests) - Порядок применения с зависимостями - Критерии приёмки для каждого шага - Стратегию роллбэка - Финальную проверку

Внедрение выполняется разработчиком по этой инструкции.
Документ — ТЗ, не код.